Network Packet Sniffer with Anomaly Detection System

Real-time Network Traffic Analysis and Threat Detection

ABSTRACT

This project presents a comprehensive network packet sniffer with real-time anomaly detection capabilities. The system captures and analyzes network traffic, detecting malicious activities such as port scanning and packet flooding attacks. It features both command-line and graphical user interfaces, SQLite database logging, automated email alerting, and visual analytics. The tool serves as an educational platform for understanding network security monitoring and intrusion detection systems while providing practical functionality for network administrators.

1. INTRODUCTION

Network security monitoring is critical in today's threat landscape. This project implements a packet sniffer that captures network traffic at the packet level and performs real-time analysis to identify suspicious activities. The system addresses the need for accessible, educational network monitoring tools that demonstrate fundamental security concepts while providing practical functionality. By combining packet capture, anomaly detection, and comprehensive logging, the tool enables both learning and operational security monitoring.

2. TOOLS & TECHNOLOGIES USED

Technology	Purpose	Version
Python 3	Core programming language	3.8+
Scapy	Packet capture and analysis	2.7.0
SQLite	Database for packet/anomaly logging	3.x
Matplotlib	Traffic visualization and charts	3.10.8
Tkinter	GUI development	Built-in
ReportLab	PDF report generation	4.4.10
SMTP	Email alerting system	Built-in

3. KEY FEATURES

- **Real-time Packet Capture:** Captures TCP, UDP, ICMP, and other protocols with detailed header analysis.
- **Anomaly Detection:** Identifies port scanning (10+ unique ports) and packet flooding (100+ packets/10s) attacks.
- **Database Logging:** Persistent storage of all packets and detected anomalies in SQLite.
- **Dual Interface:** CLI for servers and GUI with live traffic graphs for desktop monitoring.
- **Email Alerts:** Configurable SMTP-based alerting on threshold breaches.
- **Visual Analytics:** Comprehensive charts showing protocol distribution, top talkers, and port statistics.

4. IMPLEMENTATION STEPS

Step 1: Environment Setup
Installed required Python packages (scapy, matplotlib) and configured database schema with tables for packets and anomalies. Created configuration files for email alerting system.

Step 2: Packet Capture Module
Implemented core packet sniffing using Scapy library. Developed packet processor to extract IP headers, port numbers, protocols, and TCP flags. Added multi-threading for concurrent capture and analysis.

Step 3: Anomaly Detection Engine
Built detection algorithms for port scanning (tracking unique ports per source IP) and packet flooding (monitoring packet rate using sliding time windows). Implemented severity classification (MEDIUM, HIGH, CRITICAL) based on threat level.

Step 4: Database Integration
Created SQLite database schema with proper indexing for efficient queries. Implemented logging functions for both packet metadata and anomaly events with timestamp precision.

Step 5: Alerting System
Developed configurable email alerting using SMTP with HTML formatting. Created alert filtering based on anomaly type and severity levels. Added monitoring daemon for continuous anomaly checking.

Step 6: Visualization & Analytics
Built analysis module using Matplotlib for generating protocol distribution pie charts, source IP bar charts, and port statistics. Created live traffic rate graphs in GUI using real-time data updates.

Step 7: User Interfaces
Developed CLI version with command-line arguments for interface selection and packet count limits. Created GUI using Tkinter with tabbed interface for live graphs, packet logs, and alerts display. Added start/stop controls and export functionality.

5. RESULTS & OUTCOMES

The implemented system successfully captures and analyzes network traffic in real-time. Testing with simulated attacks demonstrated accurate detection of port scanning and flooding activities. The database efficiently stores thousands of packet records with fast query performance. The visualization module produces clear, informative charts for traffic analysis. The dual-interface approach makes the tool accessible for both server deployments and desktop usage. Email alerts trigger reliably when anomalies exceed configured thresholds.

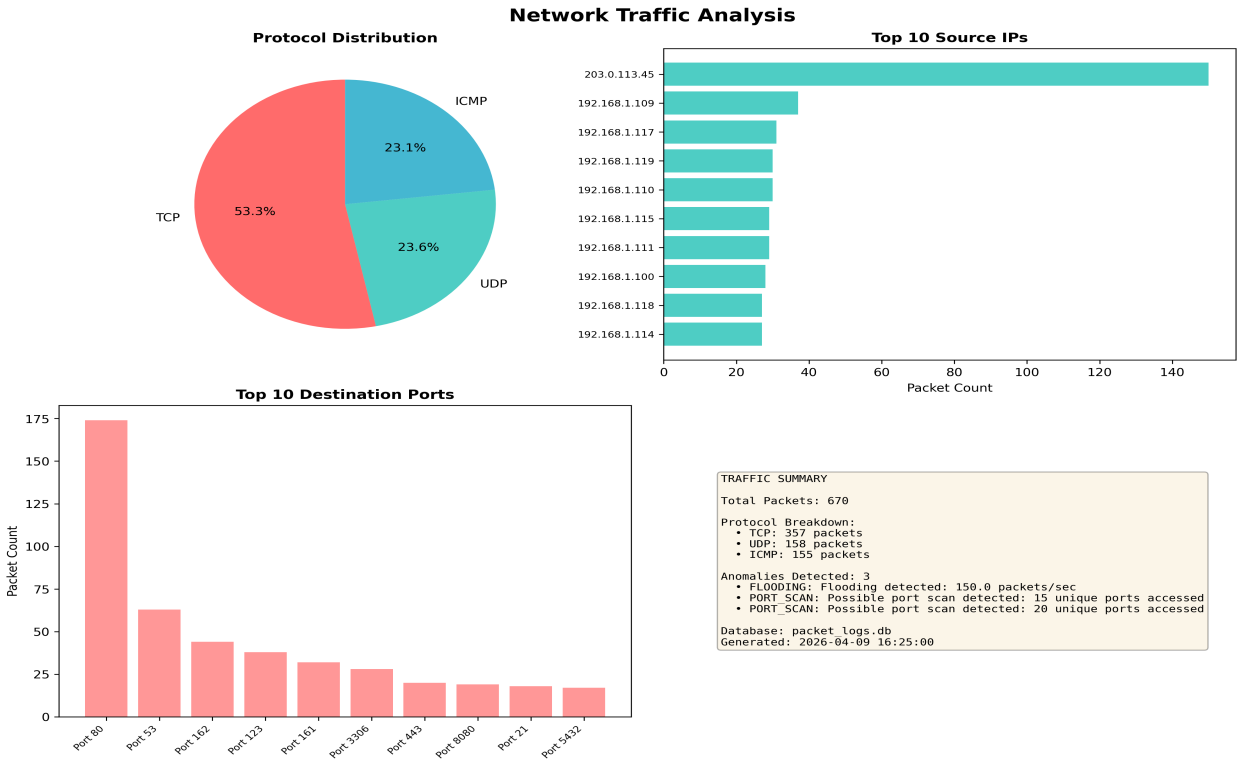

Figure 1: Traffic Analysis Dashboard showing protocol distribution, top sources, and anomalies

6. CONCLUSION

This project successfully demonstrates a functional network packet sniffer with intelligent anomaly detection. The system provides valuable insights into network traffic patterns while identifying potential security threats. The modular architecture allows for easy extension with additional detection algorithms or protocols. Future enhancements could include machine learning-based anomaly detection, GeoIP tracking, protocol-specific deep packet inspection, and integration with SIEM systems. The tool serves both educational purposes and practical network monitoring applications, making network security concepts accessible and actionable.